Africa Credit Hub — Bug Fix Guide

> **Platform:** Africa Credit Hub (Ecommerce/Fintech)
> **Stack:** React / Next.js with i18n internationalization
> **Environment:** Replit
> **Date:** April 2, 2026
> **Source:** QA Test Report Findings

Table of Contents

---

1. CRITICAL — Signup Validation Missing

Description

The signup form accepts invalid email formats (e.g., `user@`, `plaintext`, `@domain`) and passwords as short as 3 characters. This is a **security-critical** defect — it allows creation of accounts with credentials that will cause downstream failures (password reset, email verification, brute-force vulnerability).

Root Cause Analysis

The signup form either lacks client-side validation entirely, or relies solely on uncontrolled HTML5 `type="email"` validation (which browsers implement inconsistently). There is no schema-level or server-side validation enforcing password complexity or email format.

Step-by-Step Fix

1.1 Install a validation library (if not already present)

```
npm install zod
# or, if the project already uses yup/formik:
npm install yup
```

1.2 Create a shared validation schema

Create a new file for reusable auth validation:

```ts
// src/lib/validations/auth.ts
import { z } from "zod";

export const signupSchema = z.object({
  email: z
    .string()
    .min(1, "Email is required")
    .email("Please enter a valid email address"),
  password: z
    .string()
    .min(8, "Password must be at least 8 characters")
    .max(128, "Password must not exceed 128 characters")
    .regex(/[A-Z]/, "Password must contain at least one uppercase letter")
    .regex(/[a-z]/, "Password must contain at least one lowercase letter")
    .regex(/[0-9]/, "Password must contain at least one number")
    .regex(/[^A-Za-z0-9]/, "Password must contain at least one special character"),
  confirmPassword: z.string().min(1, "Please confirm your password"),
}).refine((data) => data.password === data.confirmPassword, {
  message: "Passwords do not match",
  path: ["confirmPassword"],
});

export type SignupFormData = z.infer<typeof signupSchema>;
```

1.3 Apply validation in the Signup component

```tsx
// src/components/SignupForm.tsx  (or wherever the signup form lives)
import { useState } from "react";
import { signupSchema, type SignupFormData } from "@/lib/validations/auth";
import { ZodError } from "zod";

export default function SignupForm() {
  const [formData, setFormData] = useState({ email: "", password: "",
confirmPassword: "" });
  const [errors, setErrors] = useState<Record<string, string>>({});

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
    // Clear field-level error on change
    if (errors[name]) {
      setErrors((prev) => {
        const next = { ...prev };
        delete next[name];
        return next;
      });
    }
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrors({});

    try {
      const validated = signupSchema.parse(formData);
      // Proceed with API call using `validated` data
      await fetch("/api/auth/signup", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(validated),
      });
    } catch (err) {
      if (err instanceof ZodError) {
        const fieldErrors: Record<string, string> = {};
        err.errors.forEach((e) => {
          const field = e.path.join(".");
          if (!fieldErrors[field]) fieldErrors[field] = e.message;
        });
        setErrors(fieldErrors);
      }
    }
  };

  return (
    <form onSubmit={handleSubmit} noValidate>
      <div>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          name="email"
          type="email"
          value={formData.email}
          onChange={handleChange}
          aria-invalid={!!errors.email}
          aria-describedby={errors.email ? "email-error" : undefined}
        />
        {errors.email && <p id="email-error" className="text-red-500 text-sm">{er-
rors.email}</p>}
```

```
      </div>

      <div>
        <label htmlFor="password">Password</label>
        <input
          id="password"
          name="password"
          type="password"
          value={formData.password}
          onChange={handleChange}
          aria-invalid={!!errors.password}
          aria-describedby={errors.password ? "password-error" : undefined}
        />
        {errors.password && <p id="password-error" className="text-red-500 text-sm">{e
rrors.password}</p>}
      </div>

      <div>
        <label htmlFor="confirmPassword">Confirm Password</label>
        <input
          id="confirmPassword"
          name="confirmPassword"
          type="password"
          value={formData.confirmPassword}
          onChange={handleChange}
          aria-invalid={!!errors.confirmPassword}
          aria-describedby={errors.confirmPassword ? "confirm-error" : undefined}
        />
        {errors.confirmPassword && (
          <p id="confirm-error" className="text-red-500 text-sm">{er-
rors.confirmPassword}</p>
        )}
      </div>

      <button type="submit">Sign Up</button>
    </form>
  );
}
```

1.4 Add server-side validation (API route)

Never trust client-side validation alone. Re-validate on the server:

```ts
// src/app/api/auth/signup/route.ts  (Next.js App Router)
// or src/pages/api/auth/signup.ts   (Pages Router)
import { NextResponse } from "next/server";
import { signupSchema } from "@/lib/validations/auth";

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const validated = signupSchema.parse(body);

    // ... hash password, create user in DB, etc.

    return NextResponse.json({ success: true }, { status: 201 });
  } catch (err) {
    if (err instanceof Error && err.name === "ZodError") {
      return NextResponse.json(
        { error: "Validation failed", details: (err as any).errors },
        { status: 400 }
      );
    }
    return NextResponse.json({ error: "Internal server error" }, { status: 500 });
  }
}
```

Best Practices

- **Always validate on both client AND server.** Client-side is for UX; server-side is for security.
- Use `noValidate` on `<form>` to disable inconsistent browser-native validation and rely entirely on your schema.
- Consider adding a password-strength meter component for better UX.
- Log failed validation attempts server-side for abuse monitoring.

Testing Steps

#	Action	Expected Result
1	Enter `user@` in email field and submit	Error: “Please enter a valid email address”
2	Enter `test` in email field and submit	Error: “Please enter a valid email address”
3	Enter valid email, `abc` as password	Error: “Password must be at least 8 characters”
4	Enter valid email, `abcdefgh` (no uppercase/number/special)	Error about missing character types
5	Enter valid email, strong password, mismatched confirm	Error: “Passwords do not match”
6	Enter all valid inputs	Form submits successfully
7	Send malformed JSON directly to `/api/auth/signup` via curl	400 response with validation errors

---

2. CRITICAL — Currency Display Shows “null”

Description

On borrower detail pages (`/borrowers/{id}`), the “Total Outstanding” header displays the word **“null”** instead of a currency code (e.g., `USD` , `KES` , `NGN`). For example: `null 12,500.00` instead of `KES 12,500.00` .

Root Cause Analysis

The currency value is fetched from the borrower or loan object (e.g., `borrower.currency` or `loan.currency`), but this field is either:

1. **Not populated in the API response** (the backend doesn’t return it), or
2. **Not included in the database query / ORM select**, or
3. **The template uses string interpolation without a fallback**, rendering JavaScript’s `null` as the literal string `"null"` .

The most common pattern causing this:

```
// BUG: If borrower.currency is null/undefined, this renders "null 12,500.00"
<h2>{borrower.currency} {formatNumber(borrower.totalOutstanding)}</h2>
```

Step-by-Step Fix

2.1 Add a safe currency display utility

```
// src/lib/utils/currency.ts

// Default currency for the platform (adjust to your business default)
const DEFAULT_CURRENCY = "USD";

/**
 * Safely returns a currency code, never "null" or "undefined".
 */
export function safeCurrency(currency: string | null | undefined): string {
  return currency?.trim() || DEFAULT_CURRENCY;
}

/**
 * Formats an amount with its currency code.
 * Uses Intl.NumberFormat for locale-aware formatting.
 */
export function formatCurrencyAmount(
  amount: number | null | undefined,
  currency: string | null | undefined,
  locale: string = "en-US"
): string {
  const code = safeCurrency(currency);
  const value = amount ?? 0;

  try {
    return new Intl.NumberFormat(locale, {
      style: "currency",
      currency: code,
      minimumFractionDigits: 2,
    }).format(value);
  } catch {
    // Fallback if the currency code is not recognized by Intl
    return `${code} ${value.toLocaleString(locale, { minimumFractionDigits: 2 })}`;
  }
}
```

2.2 Fix the borrower detail component

```
// src/components/BorrowerDetail.tsx (or equivalent)
import { formatCurrencyAmount } from "@/lib/utils/currency";

export default function BorrowerDetail({ borrower }: { borrower: Borrower }) {
  return (
    <div>
      <h2>Total Outstanding</h2>
      {/* BEFORE (buggy): */}
      {/* <p>{borrower.currency} {borrower.totalOutstanding?.toLocaleString()}</p> */}

      {/* AFTER (safe): */}
      <p>{formatCurrencyAmount(borrower.totalOutstanding, borrower.currency)}</p>
    </div>
  );
}
```

2.3 Ensure the backend returns the currency field

Check the API endpoint or database query that populates the borrower detail page. Ensure `currency` is included:

```
// Example: Prisma query
const borrower = await prisma.borrower.findUnique({
  where: { id: params.id },
  select: {
    id: true,
    name: true,
    totalOutstanding: true,
    currency: true,          // <-- Ensure this is selected
    // ... other fields
  },
});
```

If the `currency` field doesn't exist on the borrower model, check if it's on a related entity (e.g., `loan.currency` or `institution.defaultCurrency`) and join accordingly.

Best Practices

- **Never interpolate potentially nullable values directly into UI strings.** Always use a helper function or nullish coalescing (`??`).
- Centralize all currency formatting into a single utility so changes propagate platform-wide.
- Consider using the `Intl.NumberFormat` API with `style: "currency"` for proper locale-aware formatting (e.g., `$1,250.00` vs `1.250,00 €`).

Testing Steps

#	Action	Expected Result
1	Navigate to `/borrowers/{id}` for a borrower with a set currency	Shows correct currency code (e.g., `KES 12,500.00`)
2	Navigate to a borrower where `currency` is null in DB	Shows default currency (e.g., `USD 12,500.00`), **not** "null"
3	Navigate to a borrower where `totalOutstanding` is null	Shows `USD 0.00` or equivalent, no crash
4	Check the API response for `/api/borrowers/{id}`	Response JSON includes a `currency` field

---

3. HIGH — Broken Terms of Service & Privacy Policy Links

Description

Footer links for “Terms of Service” and “Privacy Policy” both redirect to `/security` instead of pointing to dedicated legal documentation pages (e.g., `/terms` and `/privacy`).

Root Cause Analysis

The `href` values in the footer component are incorrectly set to `/security`. This is likely a copy-paste error from when the Security page link was added, or a placeholder that was never updated.

Step-by-Step Fix

3.1 Create the legal pages

```
// src/app/terms/page.tsx  (Next.js App Router)
export default function TermsOfService() {
  return (
    <main className="max-w-4xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Terms of Service</h1>
      <p className="text-sm text-gray-500 mb-8">Last updated: April 2, 2026</p>

      <section className="space-y-4">
        <h2 className="text-xl font-semibold">1. Acceptance of Terms</h2>
        <p>
          By accessing or using the Africa Credit Hub platform, you agree to be bound
          by these Terms of Service...
        </p>
        {/* Add all relevant legal sections */}
      </section>
    </main>
  );
}
```

```
// src/app/privacy/page.tsx
export default function PrivacyPolicy() {
  return (
    <main className="max-w-4xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Privacy Policy</h1>
      <p className="text-sm text-gray-500 mb-8">Last updated: April 2, 2026</p>

      <section className="space-y-4">
        <h2 className="text-xl font-semibold">1. Information We Collect</h2>
        <p>
          Africa Credit Hub collects information you provide directly, such as when
          you create an account, apply for credit, or contact us...
        </p>
        {/* Add all relevant privacy sections */}
      </section>
    </main>
  );
}
```

3.2 Fix the footer links

Locate the footer component (typically `src/components/Footer.tsx` or `src/components/layout/Footer.tsx`):

```
// src/components/Footer.tsx

// BEFORE (buggy):
// <Link href="/security">Terms of Service</Link>
// <Link href="/security">Privacy Policy</Link>

// AFTER (fixed):
<Link href="/terms">Terms of Service</Link>
<Link href="/privacy">Privacy Policy</Link>
<Link href="/security">Security</Link>  {/* Keep this as-is */}
```

3.3 Centralize footer link configuration (recommended)

To prevent this class of bug in the future, define footer links in a single config:

```
// src/config/navigation.ts
export const footerLinks = {
  legal: [
    { label: "Terms of Service", href: "/terms" },
    { label: "Privacy Policy", href: "/privacy" },
    { label: "Security", href: "/security" },
  ],
  // ... other footer link groups
};
```

```
// In Footer.tsx:
import { footerLinks } from "@/config/navigation";

{footerLinks.legal.map((link) => (
  <Link key={link.href} href={link.href}>
    {link.label}
  </Link>
))}
```

Best Practices

- Keep legal page content in a CMS or markdown files for easy updates by non-developers.
- Add `<meta>` tags and proper `<title>` to legal pages for SEO and accessibility.
- Consider versioning legal documents (show effective date, link to previous versions).

Testing Steps

#	Action	Expected Result
1	Click “Terms of Service” in footer from any page	Navigates to `/terms`, displays ToS content
2	Click “Privacy Policy” in footer from any page	Navigates to `/privacy`, displays privacy content
3	Click “Security” in footer	Still navigates to `/security` (unchanged)
4	Directly visit `/terms` and `/privacy`	Pages render correctly, no 404

---

4. HIGH — Incomplete Language Translations (Arabic & Swahili)

Description

When users switch the language to **Arabic (ar)** or **Swahili (sw)**, the UI text does not change to the translated language — it continues to display English text. However, selecting Arabic correctly triggers a **right-to-left (RTL)** layout shift, confirming the locale switching mechanism works partially.

Root Cause Analysis

The i18n framework is correctly detecting the locale and applying layout direction, but the **translation JSON files for Arabic and Swahili are either empty, missing, or contain only a subset of keys**. The i18n library falls back to the default language (English) when a key is not found in the target locale’s file.

Step-by-Step Fix

4.1 Audit existing translation files

Check the translation directory structure. It typically looks like:

```
public/
  locales/
    en/
      common.json
      auth.json
      dashboard.json
    ar/
      common.json       ← likely empty or incomplete
    sw/
      common.json       ← likely empty or incomplete
```

Run this quick audit script to find missing keys:

```
# Compare keys between English and Arabic/Swahili
node -e "
const en = require('./public/locales/en/common.json');
const ar = require('./public/locales/ar/common.json');
const sw = require('./public/locales/sw/common.json');

const enKeys = Object.keys(en);
const missingAr = enKeys.filter(k => !(k in ar));
const missingSw = enKeys.filter(k => !(k in sw));

console.log('Missing in Arabic:', missingAr.length, 'keys');
console.log('Missing in Swahili:', missingSw.length, 'keys');
if (missingAr.length) console.log('Sample:', missingAr.slice(0, 10));
if (missingSw.length) console.log('Sample:', missingSw.slice(0, 10));
"
```

4.2 Populate the Arabic translation file

```
// public/locales/ar/common.json
{
  "nav.home": "الرئيسية",
  "nav.pricing": "الأسعار",
  "nav.security": "الأمان",
  "nav.login": "تسجيل الدخول",
  "nav.signup": "إنشاء حساب",
  "dashboard.title": "لوحة التحكم",
  "dashboard.totalOutstanding": "إجمالي المبالغ المستحقة",
  "borrowers.title": "المقترضون",
  "institutions.title": "المؤسسات",
  "institutions.types.regulator": "جهة تنظيمية",
  "institutions.types.bank": "بنك",
  "institutions.types.microfinance": "تمويل أصغر",
  "footer.terms": "شروط الخدمة",
  "footer.privacy": "سياسة الخصوصية",
  "footer.security": "الأمان"
  // ... complete all keys from the English file
}
```

4.3 Populate the Swahili translation file

```
// public/locales/sw/common.json
{
  "nav.home": "Nyumbani",
  "nav.pricing": "Bei",
  "nav.security": "Usalama",
  "nav.login": "Ingia",
  "nav.signup": "Jisajili",
  "dashboard.title": "Dashibodi",
  "dashboard.totalOutstanding": "Jumla ya Deni",
  "borrowers.title": "Wakopaji",
  "institutions.title": "Taasisi",
  "institutions.types.regulator": "Mdhibiti",
  "institutions.types.bank": "Benki",
  "institutions.types.microfinance": "Fedha Ndogo",
  "footer.terms": "Masharti ya Huduma",
  "footer.privacy": "Sera ya Faragha",
  "footer.security": "Usalama"
  // ... complete all keys from the English file
}
```

4.4 Ensure next-i18next config includes the locales

```
// next-i18next.config.js
module.exports = {
  i18n: {
    defaultLocale: "en",
    locales: ["en", "fr", "ar", "sw"],
    localeDetection: false,     // Prevent auto-detection overriding user choice
  },
  fallbackLng: "en",           // Fallback if key is missing
  ns: ["common", "auth", "dashboard"],
  defaultNS: "common",
};
```

4.5 Verify RTL handling is applied to all text elements

Arabic RTL is already working for layout, but verify that text alignment is also correct:

```
// src/app/layout.tsx  or  src/pages/_app.tsx
import { useRouter } from "next/router";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  const { locale } = useRouter();
  const dir = locale === "ar" ? "rtl" : "ltr";

  return (
    <html lang={locale} dir={dir}>
      <body>{children}</body>
    </html>
  );
}
```

Best Practices

- **Use a translation management tool** (e.g., Crowdin, Lokalise, or even a spreadsheet) to track translation completeness across locales.

- **Create a CI check** that compares keys across all locale files and fails if any are missing:
  ```bash
  # scripts/check-translations.js — run in CI
  const en = require("../public/locales/en/common.json");
  const locales = ["ar", "sw", "fr"];
  let missing = false;
  for (const loc of locales) {
    const data = require(`../public/locales/${loc}/common.json`);
    const missingKeys = Object.keys(en).filter((k) => !(k in data));
    if (missingKeys.length) {
      console.error(`${loc}: ${missingKeys.length} missing keys`);
      missing = true;
    }
  }
  if (missing) process.exit(1);
  ```
- Add `fallbackLng` configuration so missing keys at least show English rather than raw keys.

Testing Steps

#	Action	Expected Result
1	Switch language to Arabic	All UI text renders in Arabic; layout is RTL
2	Switch language to Swahili	All UI text renders in Swahili; layout is LTR
3	Switch back to English	All text reverts to English
4	Navigate to every major page in Arabic	No English text visible (except proper nouns / brand names)
5	Run `scripts/check-translations.js`	Exits with code 0, no missing keys

---

5. HIGH — Raw i18n Keys on /institutions Page

Description

The `/institutions` page displays raw translation keys instead of human-readable labels. For example, users see `"Institutions.Types.Regulator"` instead of `"Regulator"`.

Root Cause Analysis

This typically happens when:

1. The translation namespace containing the `Institutions` keys is **not loaded** on this page, or
2. The keys in the translation file **don't match** the keys used in the component (case sensitivity, nesting mismatch), or

3. The component doesn't use the `useTranslation` hook or uses the wrong namespace.

Step-by-Step Fix

5.1 Verify the translation keys exist in the English file

```
// public/locales/en/common.json  (or a dedicated institutions.json namespace)
{
  "Institutions": {
    "Title": "Institutions",
    "Description": "Partner financial institutions and regulators",
    "Types": {
      "Regulator": "Regulator",
      "Bank": "Bank",
      "Microfinance": "Microfinance Institution",
      "CreditBureau": "Credit Bureau",
      "SaccoCooperative": "SACCO / Cooperative"
    },
    "Table": {
      "Name": "Name",
      "Type": "Type",
      "Country": "Country",
      "Status": "Status"
    }
  }
}
```

5.2 Fix the component to use translations correctly

```tsx
// src/app/institutions/page.tsx  (or src/pages/institutions.tsx)
import { useTranslation } from "next-i18next";
// For App Router with app-i18next, use:
// import { useTranslation } from "react-i18next";

export default function InstitutionsPage() {
  const { t } = useTranslation("common"); // Ensure correct namespace

  return (
    <main>
      <h1>{t("Institutions.Title")}</h1>
      <p>{t("Institutions.Description")}</p>

      <table>
        <thead>
          <tr>
            <th>{t("Institutions.Table.Name")}</th>
            <th>{t("Institutions.Table.Type")}</th>
            <th>{t("Institutions.Table.Country")}</th>
            <th>{t("Institutions.Table.Status")}</th>
          </tr>
        </thead>
        <tbody>
          {institutions.map((inst) => (
            <tr key={inst.id}>
              <td>{inst.name}</td>
              {/* Dynamically look up the institution type label */}
              <td>{t(`Institutions.Types.${inst.type}`)}</td>
              <td>{inst.country}</td>
              <td>{inst.status}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </main>
  );
}
```

5.3 If using Pages Router, ensure `getStaticProps` / `getServerSideProps` loads the namespace

```tsx
// src/pages/institutions.tsx
import { serverSideTranslations } from "next-i18next/serverSideTranslations";

export async function getStaticProps({ locale }: { locale: string }) {
  return {
    props: {
      ...(await serverSideTranslations(locale, ["common"])),
      // Make sure "common" (or your namespace) is listed here
    },
  };
}
```

5.4 For App Router — ensure the i18n provider wraps the page

If using Next.js App Router, translation loading works differently. Make sure the provider is in place:

```tsx
// src/app/providers.tsx
"use client";
import { I18nextProvider } from "react-i18next";
import i18n from "@/lib/i18n"; // your i18n initialization

export function Providers({ children }: { children: React.ReactNode }) {
  return <I18nextProvider i18n={i18n}>{children}</I18nextProvider>;
}
```

Best Practices

- Use the `keySeparator` config in i18next to control how nested keys are resolved (default is `.`). If your JSON uses nested objects, ensure the separator matches.
- Add a development-mode warning that highlights raw keys visually. Some i18n libraries support this with `saveMissing: true` and `missingKeyHandler`.
- Keep key naming consistent: choose either `PascalCase.Nested` or `snake_case.nested` — don't mix.

Testing Steps

#	Action	Expected Result
1	Navigate to `/institutions` in English	All labels are human-readable ("Regulator", "Bank", etc.)
2	Navigate to `/institutions` in Arabic	Labels display in Arabic
3	Navigate to `/institutions` in Swahili	Labels display in Swahili
4	Search source code for hard-coded raw key strings in JSX	None found — all text uses `t()`

---

6. MEDIUM — Inconsistent Navigation Bars

Description

The homepage uses a different navigation bar component (or different styling/links) compared to sub-pages like `/pricing` and `/security`. This creates a disorienting experience where navigation options change unexpectedly.

Root Cause Analysis

Common causes:

- **Separate navbar components** for the landing page vs. inner pages, developed independently.
- The homepage uses a `<LandingNavbar />` while inner pages use a `<DashboardNavbar />` or `<DefaultNavbar />`.
- Different layout wrappers for different route groups.

Step-by-Step Fix

6.1 Create a single, unified Navbar component

```
// src/components/layout/Navbar.tsx
"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { useTranslation } from "react-i18next";
import { navLinks } from "@/config/navigation";

export default function Navbar() {
  const pathname = usePathname();
  const { t } = useTranslation("common");

  return (
    <header className="sticky top-0 z-50 bg-white border-b border-gray-200">
      <nav className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 flex items-center
justify-between h-16">
        {/* Logo — always the same */}
        <Link href="/" className="flex items-center gap-2">
          <img src="/logo.svg" alt="Africa Credit Hub" className="h-8" />
          <span className="font-bold text-lg">Africa Credit Hub</span>
        </Link>

        {/* Navigation links — single source of truth */}
        <div className="hidden md:flex items-center gap-6">
          {navLinks.map((link) => (
            <Link
              key={link.href}
              href={link.href}
              className={`text-sm font-medium transition-colors ${
                pathname === link.href
                  ? "text-blue-600"
                  : "text-gray-600 hover:text-gray-900"
              }`}
            >
              {t(link.labelKey)}
            </Link>
          ))}
        </div>

        {/* Auth buttons */}
        <div className="flex items-center gap-3">
          <Link href="/login" className="text-sm font-medium text-gray-600 hover:text-
gray-900">
            {t("nav.login")}
          </Link>
          <Link href="/signup" className="btn-primary text-sm">
            {t("nav.signup")}
          </Link>
        </div>
      </nav>
    </header>
  );
}
```

6.2 Define navigation links centrally

```
// src/config/navigation.ts
export const navLinks = [
  { labelKey: "nav.home", href: "/" },
  { labelKey: "nav.pricing", href: "/pricing" },
  { labelKey: "nav.security", href: "/security" },
  { labelKey: "nav.institutions", href: "/institutions" },
];
```

6.3 Use the unified Navbar in the root layout

```
// src/app/layout.tsx
import Navbar from "@/components/layout/Navbar";
import Footer from "@/components/layout/Footer";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html>
      <body>
        <Navbar />       {/* ← One navbar everywhere */}
        <main>{children}</main>
        <Footer />
      </body>
    </html>
  );
}
```

6.4 Remove old duplicate navbar components

Search the codebase for all navbar variants and remove them:

```
grep -rn "Navbar\|NavBar\|Navigation\|TopBar\|Header" src/components/ --in-
clude="*.tsx" --include="*.jsx"
```

Delete or consolidate any duplicates found. Update all imports to point to the unified `Navbar`.

Best Practices

- A single `Navbar` component should handle all states (logged-in vs. logged-out, homepage vs. sub-page) using conditional rendering, not separate components.
- If the homepage genuinely needs a different navbar style (e.g., transparent over a hero image), handle it with a `variant` prop, not a different component:
  `tsx`
  `<Navbar variant={pathname === "/" ? "transparent" : "solid"} />`

Testing Steps

#	Action	Expected Result
1	Visit homepage, note navbar links and styling	Navbar visible with all expected links
2	Navigate to `/pricing`	Identical navbar as homepage
3	Navigate to `/security`	Identical navbar as homepage
4	Navigate to `/institutions`	Identical navbar as homepage
5	Resize to mobile	Consistent mobile hamburger menu across all pages

---

7. MEDIUM — Missing 404 Error Handler

Description

Visiting any invalid URL (e.g., `/nonexistent-page`, `/asdfgh`) redirects the user to `/login` with a `200 OK` HTTP status instead of showing a proper 404 error page.

Root Cause Analysis

The application likely has a catch-all route or middleware that redirects all unmatched routes to `/login` — probably an overly broad authentication guard. There is no `not-found` page defined.

Step-by-Step Fix

7.1 Create a 404 page

Next.js App Router:

```
// src/app/not-found.tsx
import Link from "next/link";

export default function NotFound() {
  return (
    <main className="min-h-screen flex flex-col items-center justify-center text-
center px-4">
      <h1 className="text-6xl font-bold text-gray-900">404</h1>
      <p className="mt-4 text-xl text-gray-600">Page not found</p>
      <p className="mt-2 text-gray-500">
        The page you're looking for doesn't exist or has been moved.
      </p>
      <Link
        href="/"
        className="mt-8 inline-block px-6 py-3 bg-blue-600 text-white rounded-lg
hover:bg-blue-700 transition"
      >
        Go to Homepage
      </Link>
    </main>
  );
}
```

Next.js Pages Router:

```
// src/pages/404.tsx
export default function Custom404() {
  return (
    <main className="min-h-screen flex flex-col items-center justify-center text-
center px-4">
      <h1 className="text-6xl font-bold text-gray-900">404</h1>
      <p className="mt-4 text-xl text-gray-600">Page not found</p>
      <a href="/" className="mt-8 inline-block px-6 py-3 bg-blue-600 text-white
rounded-lg">
        Go to Homepage
      </a>
    </main>
  );
}
```

7.2 Fix the overly broad authentication redirect

Locate the middleware or route guard that’s catching all routes and redirecting to `/login`. It’s usually in `src/middleware.ts`:

```
// src/middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Define which routes REQUIRE authentication
const protectedRoutes = ["/dashboard", "/borrowers", "/institutions", "/settings"];
// Define public routes that should always be accessible
const publicRoutes = ["/", "/login", "/signup", "/pricing", "/security", "/terms", "/
privacy"];

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Only redirect to login if the path is a KNOWN protected route
  // and the user is not authenticated
  const isProtectedRoute = protectedRoutes.some((route) => path-
name.startsWith(route));

  if (isProtectedRoute) {
    const token = request.cookies.get("auth-token")?.value;
    if (!token) {
      const loginUrl = new URL("/login", request.url);
      loginUrl.searchParams.set("redirect", pathname);
      return NextResponse.redirect(loginUrl);
    }
  }

  // For all other routes (including unknown ones), let Next.js handle it
  // Next.js will automatically show the 404 page for unmatched routes
  return NextResponse.next();
}

export const config = {
  // Only run middleware on specific paths, NOT on everything
  matcher: ["/dashboard/:path*", "/borrowers/:path*", "/institutions/:path*", "/set-
tings/:path*"],
};
```

Best Practices

- **Never** use a catch-all redirect to `/login`. It masks errors and breaks the HTTP contract (200 for a page that doesn't exist violates HTTP semantics and confuses search engines).
- Use the `matcher` config in middleware to scope auth checks to known protected routes only.
- Return proper HTTP status codes: `404` for not found, `401` for unauthorized, `302/307` for redirects.

Testing Steps

#	Action	Expected Result
1	Visit `/this-page-does-not-exist`	See the 404 page, HTTP status is `404`
2	Visit `/asdfgh`	See the 404 page, HTTP status is `404`
3	Visit `/dashboard` while logged out	Redirect to `/login` (auth guard)
4	Visit `/pricing` while logged out	Page renders normally (public route)
5	Check response headers with `curl -I`	Confirm `404` status on invalid URLs

---

8. MEDIUM — Branding Version Inconsistencies

Description

Different pages reference different version numbers for the platform: some show **v2.0**, others **v2.1**, and others **v2.5**. This undermines trust and creates confusion about which version is current.

Root Cause Analysis

Version strings are hardcoded in multiple places across different components, rather than being sourced from a single constant or config. Different developers updated different pages at different times.

Step-by-Step Fix

8.1 Create a single source of truth for version info

```
// src/config/branding.ts
export const BRANDING = {
  name: "Africa Credit Hub",
  version: "2.5",                    // Single source of truth
  displayVersion: "v2.5",
  tagline: "Credit infrastructure for Africa",
  copyrightYear: new Date().getFullYear(),
  website: "https://africacredithub.com",
} as const;
```

8.2 Find and replace all hardcoded version strings

```
# Search for all hardcoded version references
grep -rn "v2\.0\|v2\.1\|v2\.5\|version.*2\." src/ --include="*.tsx" --include="*.jsx"
--include="*.ts" --include="*.json"
```

Replace every occurrence with the centralized constant:

```
// BEFORE (scattered across codebase):
<span>Africa Credit Hub v2.0</span>
<footer>Version 2.1</footer>
<p>Platform v2.5</p>

// AFTER (all reference the same constant):
import { BRANDING } from "@/config/branding";

<span>{BRANDING.name} {BRANDING.displayVersion}</span>
<footer>Version {BRANDING.version}</footer>
<p>Platform {BRANDING.displayVersion}</p>
```

8.3 Also update `package.json` to match

```
{
  "name": "africa-credit-hub",
  "version": "2.5.0"
}
```

Best Practices

- Consider pulling the version from `package.json` at build time so it's always in sync:
  `ts`
  `// src/config/branding.ts`
  `import packageJson from "../../package.json";`
  `export const BRANDING = {`
  `version: packageJson.version,`
  `displayVersion: \`v${packageJson.version}\`,`
  `// ...`
  `};`
- Add a linting rule or CI check that flags hardcoded version strings (regex: `/v\d+\.\d+/` in JSX).
- Use semantic versioning properly: `MAJOR.MINOR.PATCH`.

Testing Steps

#	Action	Expected Result
1	Visit homepage, check version display	Shows `v2.5`
2	Visit `/pricing`, check footer/ header	Shows `v2.5`
3	Visit `/security`, check all version refs	Shows `v2.5`
4	`grep -rn "v2\.0|v2\.1" src/`	Zero results — no stale versions remain

---

9. LOW — Minor UI/UX Issues

These are lower-priority polish items that improve overall quality but don't block functionality.

9a. Accessibility & Interaction Refinements

Common issues found in fintech platforms:

```
// Ensure all interactive elements have focus indicators
// tailwind.config.js or global CSS:

// global.css
*:focus-visible {
  outline: 2px solid #2563eb;
  outline-offset: 2px;
}

// Ensure buttons have proper hover/active states
.btn-primary {
  @apply bg-blue-600 text-white px-4 py-2 rounded-lg
         hover:bg-blue-700
         active:bg-blue-800
         focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:ring-offset-2
         disabled:opacity-50 disabled:cursor-not-allowed
         transition-colors;
}
```

9b. Loading States & Empty States

```
// Add skeleton loaders for data-heavy pages
function InstitutionsSkeleton() {
  return (
    <div className="animate-pulse space-y-4">
      {Array.from({ length: 5 }).map((_, i) => (
        <div key={i} className="h-12 bg-gray-200 rounded" />
      ))}
    </div>
  );
}

// Add meaningful empty states
function EmptyState({ message }: { message: string }) {
  return (
    <div className="text-center py-16 text-gray-500">
      <p className="text-lg">{message}</p>
    </div>
  );
}
```

9c. Mobile Responsiveness Check

Ensure all pages are tested at these breakpoints:

Breakpoint	Width	Target
`sm`	640px	Large phones
`md`	768px	Tablets
`lg`	1024px	Laptops
`xl`	1280px	Desktops

Summary Checklist

#	Bug	Priority	Status
1	Signup validation missing	🔴 CRITICAL	☐ TODO
2	Currency displays “null”	🔴 CRITICAL	☐ TODO
3	Broken ToS & Privacy links	🟠 HIGH	☐ TODO
4	Arabic & Swahili translations incomplete	🟠 HIGH	☐ TODO
5	Raw i18n keys on /institutions	🟠 HIGH	☐ TODO
6	Inconsistent navigation bars	🟡 MEDIUM	☐ TODO
7	Missing 404 handler	🟡 MEDIUM	☐ TODO
8	Version string inconsistencies	🟡 MEDIUM	☐ TODO
9	Minor UI/UX polish	🟢 LOW	☐ TODO

> **Recommended fix order:** 1 → 2 → 7 → 3 → 5 → 4 → 6 → 8 → 9
> (Critical security first, then structural issues, then polish.)

Generated for the Africa Credit Hub development team — April 2, 2026